

1. Title and Project Overview

Title: Student Management System

The **Student Management System** is a full-stack web application developed to manage student information efficiently. The system provides a simple interface for adding, viewing, updating, and deleting student records. It uses a frontend developed with **HTML, CSS, and JavaScript**, a backend developed using **Django and Django REST Framework**, and **SQLite** as the database. REST APIs are used for communication between the frontend and backend. The application also includes data validation to prevent invalid or duplicate student information.

**Student Management System**
Registry of enrolled students — add, update, and track records

New Entry

Student name

e.g. Supriya

Email

e.g. supriya@gmail.com

Phone number

10-digit number

Department

Select

Year

Select

Student Records

4

Search by name, email or department.

ID	Name	Email	Phone	Dept	Year	Gender	Address	Actions
5	arunkumar S	arunkumar@gmail.com	7890678906	CIVIL	3	Male	madurai	Edit Delete
4	Udhaya	udhaya@gmail.com	1234567789	CSE	2	Female	Cbe	Edit Delete
3	supriya M	supriya@gmail.com	8790087900	CSE	2	Female	vellakovil,tiruppur	Edit Delete
2	rithika shree K	rithikashree@gmail.com	9876509876	CSE	2	Female	ottansathiram	Edit Delete

2. Problem Statement

Managing student information manually using notebooks or spreadsheets can be time-consuming and may lead to duplicate, incorrect, or lost records. It can also make updating and deleting student information difficult.

The proposed Student Management System provides a centralized web-based solution for storing and managing student records. It allows users to perform CRUD operations and access student information through REST APIs.

3. Objectives

The main objectives of the project are:

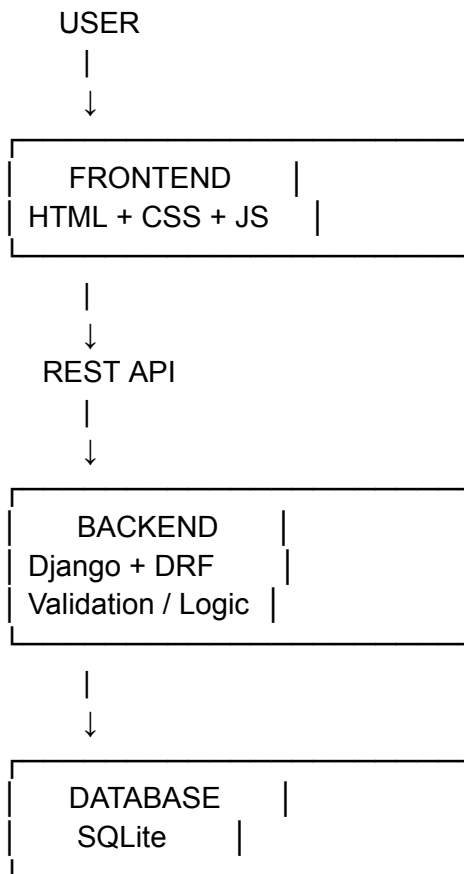
- To develop a user-friendly Student Management System.
 - To store student information in a structured database.
 - To implement **Create, Read, Update, and Delete (CRUD)** operations.
 - To provide REST APIs for student management.
 - To validate student information before storing it.
 - To prevent duplicate email records.
 - To connect the frontend with the Django backend.
 - To test API functionality using Postman.
 - To provide a reliable and easy-to-use system for managing student records.
-

4. Technology Stack

Component	Technology
Frontend	HTML5, CSS3, JavaScript
Backend	Python, Django
REST API	Django REST Framework
Database	SQLite
API Testing	Postman
Code Editor	Visual Studio Code
Version Control	Git
Repository	GitHub

5. System Architecture

The system follows a **three-layer architecture**.



Working Process

1. The user enters student information through the frontend.
 2. JavaScript sends the request to the Django REST API.
 3. Django receives and validates the data.
 4. The validated data is stored in the SQLite database.
 5. The backend sends a response to the frontend.
 6. The frontend displays the updated student information.
-

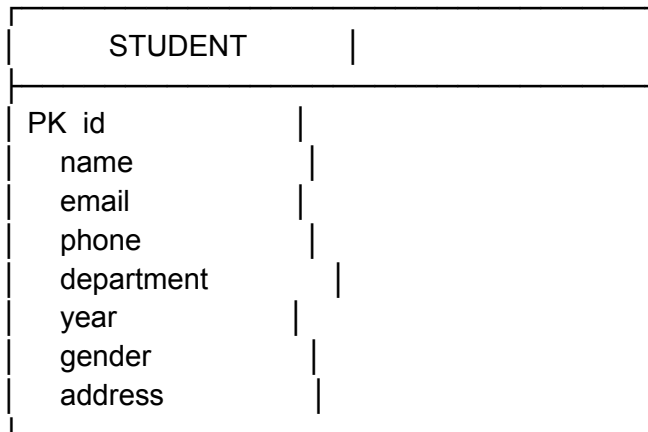
6. Database / ER Diagram

The main database entity is **Student**.

Student Entity

Field	Description
id	Unique student identifier
name	Student name
email	Student email
phone	Student phone number
department	Student department
year	Current academic year
gender	Student gender
address	Student address

ER Diagram



The **id** field uniquely identifies each student record.

7. UI Screenshots

The following screenshots can be included in the project documentation.

Screenshot 1 – Student Management Home Page.

SMS

Student Management System

Registry of enrolled students — add, update, and track records

New Entry

Student name

e.g. Supriya

Email

e.g. supriya@gmail.com

Phone number

10-digit number

Department

Select

Year

Select

Student Records 4

Search by name, email or department.

ID	Name	Email	Phone	Dept	Year	Gender	Address	Actions
5	arunkumar S	arunkumar@gmail.com	7890678906	CIVIL	3	Male	madurai	<a>Edit <a>Delete
4	Udhaya	udhaya@gmail.com	1234567789	CSE	2	Female	Cbe	<a>Edit <a>Delete
3	supriya M	supriya@gmail.com	8790087900	CSE	2	Female	vellakovil,tiruppur	<a>Edit <a>Delete
2	rithika shree K	rithikashree@gmail.com	9876509876	CSE	2	Female	ottansathiram	<a>Edit <a>Delete

Screenshot 2 – Add Student

Email

e.g. supriya@gmail.com

Phone number

10-digit number

Department

Select

Year

Select

Gender

☐ Male ☐ Female ☐ Other

Address

e.g. Madurai

Add student

S

4	Udhaya	udhaya@gmail.com	1234567789	CSE	2	Female	Cbe	Edit	Delete
3	supriya M	supriya@gmail.com	8790087900	CSE	2	Female	vellakovil,tiruppur	Edit	Delete
2	rithika shree K	rithikashree@gmail.com	9876509876	CSE	2	Female	ottansathiram	Edit	Delete

Screenshot 3 – Student List

Email

e.g. supriya@gmail.com

Phone number

10-digit number

Department

Select

Year

Select

Gender

☐ Male ☐ Female ☐ Other

Address

e.g. Madurai

Add student

S

4	Udhaya	udhaya@gmail.com	1234567789	CSE	2	Female	Cbe	Edit	Delete
3	supriya M	supriya@gmail.com	8790087900	CSE	2	Female	vellakovil,tiruppur	Edit	Delete
2	rithika shree K	rithikashree@gmail.com	9876509876	CSE	2	Female	ottansathiram	Edit	Delete

Screenshot 4 – Update Student

Email

e.g. supriya@gmail.com

Phone number

10-digit number

Department

Select

Year

Select

Gender

☐ Male ☐ Female ☐ Other

Address

e.g. Madurai

Add student

5									
4	Udhaya	udhaya@gmail.com	1234567789	CSE	2	Female	Cbe	Edit	Delete
3	supriya M	supriya@gmail.com	8790087900	CSE	2	Female	vellakovil,tiruppur	Edit	Delete
2	rithika shree K	rithikashree@gmail.com	9876509876	CSE	2	Female	ottansathiram	Edit	Delete

Screenshot 5 – Delete Student

Email

e.g. supriya@gmail.com

Phone number

10-digit number

Department

Select

Year

Select

Gender

☐ Male ☐ Female ☐ Other

Address

e.g. Madurai

Add student

5									
4	Udhaya	udhaya@gmail.com	1234567789	CSE	2	Female	Cbe	Edit	Delete
3	supriya M	supriya@gmail.com	8790087900	CSE	2	Female	vellakovil,tiruppur	Edit	Delete
2	rithika shree K	rithikashree@gmail.com	9876509876	CSE	2	Female	ottansathiram	Edit	Delete

Screenshot 6 – Validation

New Entry

Student name

aarthi

Email

23678

Enter a valid email address.

Phone number

90987

Phone number must be exactly 10 digits.

Department

EEE

Year

3

Gender

☐ Male ☒ Female ☐ Other

Address

chennai

Student Records 4

Search by name, email or department.

ID	Name	Email	Phone	Dept	Year	Gender	Address	Actions
5	arunkumar S	arunkumar@gmail.com	7890678906	CIVIL	3	Male	madurai	Edit Delete
4	Udhaya	udhaya@gmail.com	1234567789	CSE	2	Female	Cbe	Edit Delete
3	supriya M	supriya@gmail.com	8790087900	CSE	2	Female	vellakovil,tiruppur	Edit Delete
2	rithika shree K	rithikashree@gmail.com	9876509876	CSE	2	Female	ottansathiram	Edit Delete

8. API Endpoint Documentation

The Student Management System provides REST API endpoints for managing student records.

Base URL

http://127.0.0.1:8000

1. Get All Students

Method: GET

/api/students/

Purpose: Retrieves all student records.

Expected Response:

```
[
  {
    "id": 1,
    "name": "Arun Kumar",
    "email": "arun@gmail.com",
    "phone": "9876543210",
    "department": "CSE",
```

```
    "year": 2,  
    "gender": "Male",  
    "address": "Chennai"  
  }  
]
```

2. Create Student

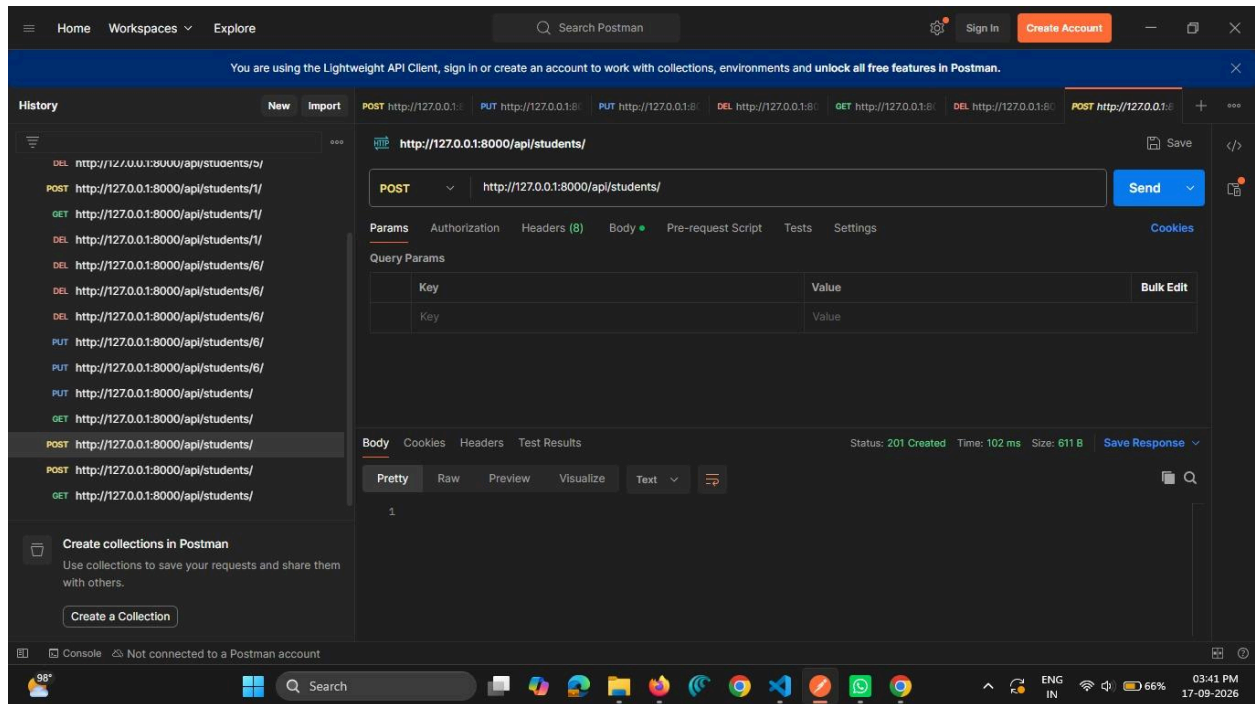
Method: POST

/api/students/

Request Body:

```
{  
  "name": "Arun Kumar",  
  "email": "arun@gmail.com",  
  "phone": "9876543210",  
  "department": "CSE",  
  "year": 2,  
  "gender": "Male",  
  "address": "Chennai"  
}
```

Expected Status: 201 Created



3. Get Student by ID

Method: **GET**

/api/students/{id}/

Purpose: Retrieves a specific student record.

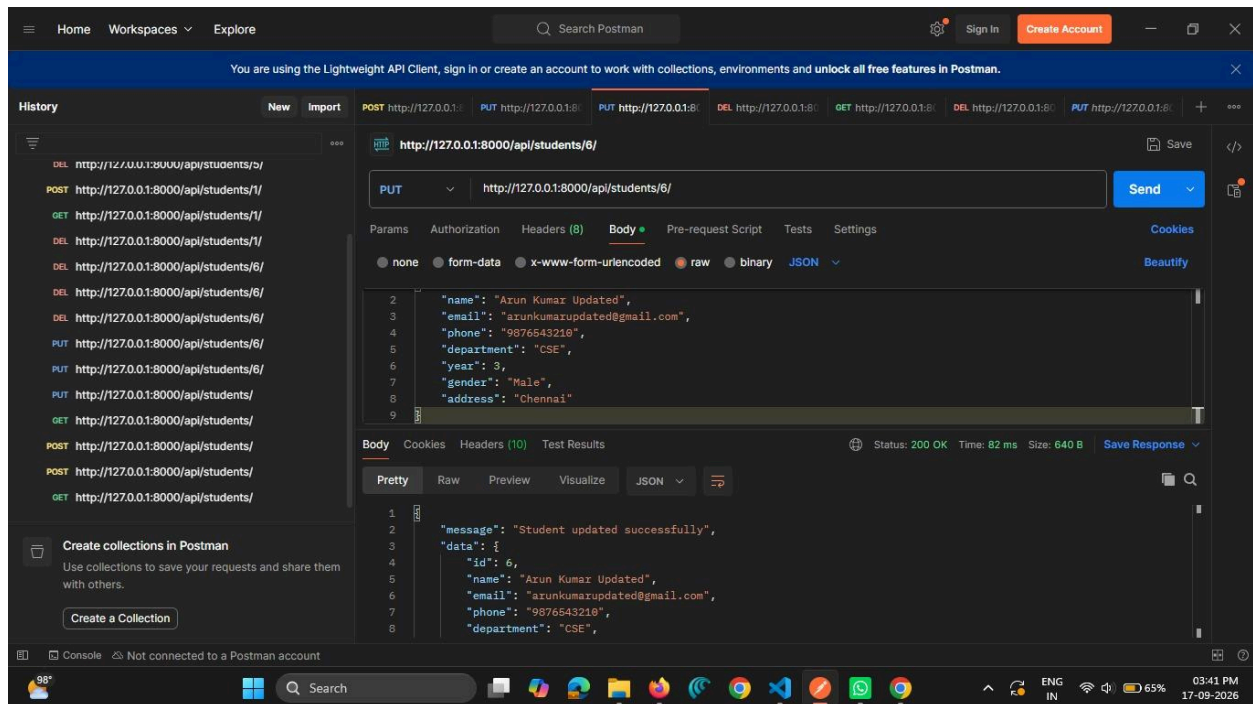
4. Update Student

Method: **PUT**

/api/students/{id}/

Purpose: Updates an existing student record.

Expected Status: 200 OK



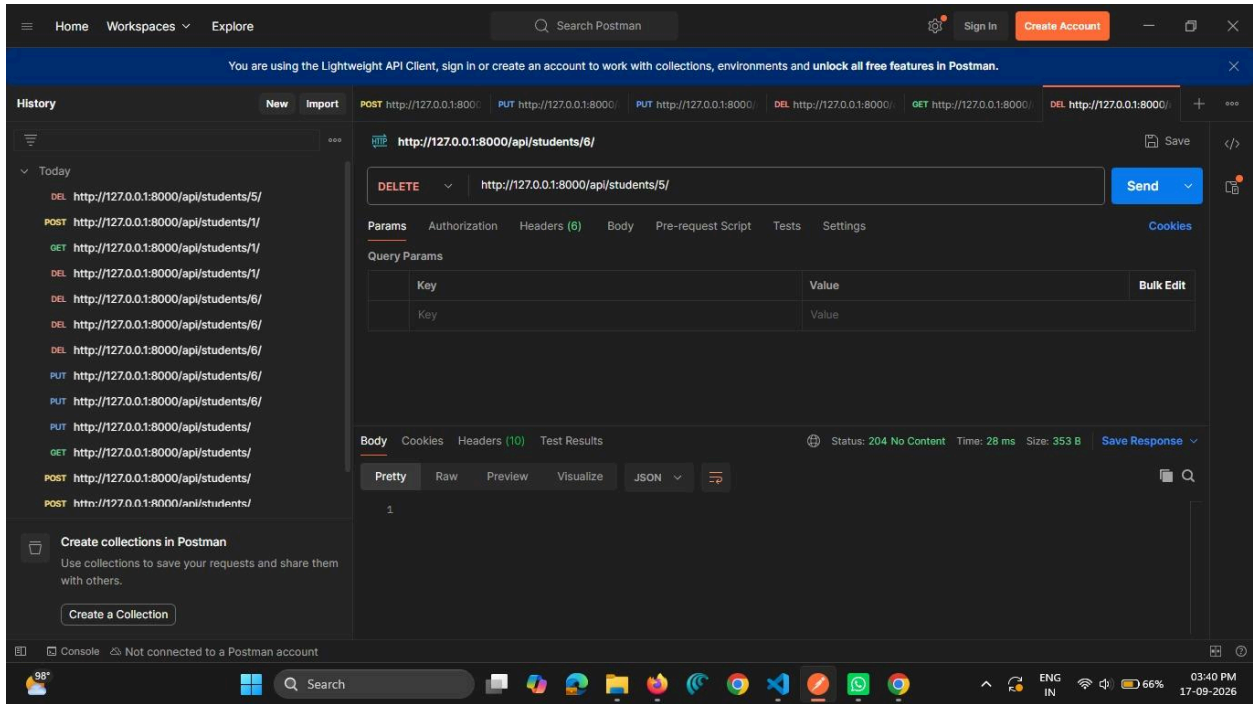
5. Delete Student

Method: DELETE

/api/students/{id}/

Purpose: Deletes a student record.

Expected Status: 204 No Content



9. CRUD Implementation Details

The application implements all four basic database operations.

Create

The **POST** API is used to create a new student record.

POST /api/students/

The backend validates the submitted information before storing it in the database.

Read

The **GET** API retrieves student information.

GET /api/students/

It can retrieve all students or a particular student using the student ID.

Update

The **PUT** API modifies an existing student record.

PUT /api/students/{id}/

The updated information is validated before saving.

Delete

The **DELETE** API removes a student record.

DELETE /api/students/{id}/

After successful deletion, the API returns **204 No Content**.

10. Testing Results

API testing was performed using **Postman**.

Test	Method	Expected Result	Status
Get students	GET	Student list displayed	Passed
Add student	POST	Student created	Passed
Duplicate email	POST	Validation error	Passed
Update student	PUT	Student updated	Tested
Invalid student ID	PUT	Student not found	Tested
Delete student	DELETE	Student deleted	To be captured
Get after deletion	GET	Updated student list	To be captured

Example Validation

When an existing email is submitted, the API returns an error such as:

```
{  
  "email": [  
    "student with this email already exists."  
  ]  
}
```

This prevents duplicate student records.

11. Installation and Execution Steps

Step 1 – Open the project

```
cd C:\Users\ELCOT\Downloads\StudentManagementSystem
```

Step 2 – Activate virtual environment

```
.\env\Scripts\Activate.ps1
```

Step 3 – Go to backend

```
cd backend
```

Step 4 – Install required packages

```
pip install django django-rest-framework django-cors-headers
```

Step 5 – Apply migrations

```
python manage.py migrate
```

Step 6 – Start Django server

```
python manage.py runserver
```

The backend will run at:

<http://127.0.0.1:8000/>

API:

<http://127.0.0.1:8000/api/students/>

Step 7 – Run Frontend

Open the **frontend** folder in VS Code and run **index.html** using **Live Server**.

The frontend and backend should both be running for complete application functionality.

12. Challenges and Solutions

Challenge 1 – Virtual Environment Activation

Problem: The virtual environment could not initially be activated.

Solution: The virtual environment was recreated and PowerShell execution policy was adjusted for the current session.

Challenge 2 – API Testing with Postman

Problem: Postman initially could not access the local Django server when the Cloud Agent was being used.

Solution: The Postman Desktop Agent was used to communicate with the local API.

Challenge 3 – Duplicate Email

Problem: Updating or creating a student using an already registered email generated a validation error.

Solution: Email uniqueness validation was maintained and a unique email was used for new/update records.

Challenge 4 – Student Not Found

Problem: An incorrect student ID resulted in:

```
{  
  "error": "Student not found"  
}
```

Solution: The GET API was used to identify the correct student ID before performing PUT or DELETE operations.

Challenge 5 – GitHub Push Conflict

Problem: The local project and GitHub repository contained different README files.

Solution: The README merge conflict was resolved and the changes were committed before pushing the project to GitHub.

13. Future Enhancements

The following features can be added in future versions:

- Student login and authentication.
 - Admin dashboard.
 - Search and filter functionality.
 - Pagination for large student datasets.
 - Student profile pages.
 - Attendance management.
 - Marks and grade management.
 - Export student information to PDF/Excel.
 - PostgreSQL or MySQL database support.
 - Deployment to a cloud platform.
 - Improved responsive design for mobile devices.
 - Role-based access control.
-

14. Git Repository / Reference Details

The project source code is maintained using **Git** and **GitHub**.

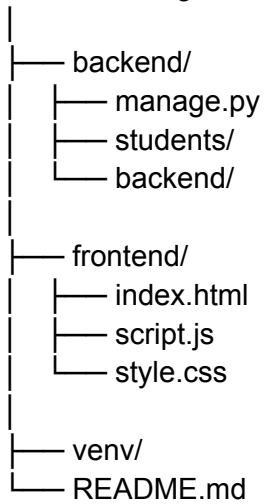
GitHub Repository

Student Management System

github.com/supriya261007-eng/Student-Management-System

The repository contains the project source code, including:

StudentManagementSystem/



Git is used for version control and GitHub is used to maintain and share the project source code.

Conclusion

The **Student Management System** provides a simple full-stack solution for managing student records. It successfully demonstrates frontend development, Django backend development, REST API communication, database operations, CRUD functionality, validation, API testing, and Git-based version control. The project can be further enhanced with authentication, advanced student management features, and cloud deployment.